

Introduction to Deep Learning

Assignment 0

Pelagia Kalpakidou(3744825), Myriana Miltiadous(3699463), Elena Pitta(3840220)

Introduction

The objective of this assignment is to develop and evaluate several algorithms for classifying images of handwritten digits and designing a neural network from scratch for the XOR problem. The dataset used is a simplified version of MNIST. The data is split into a training set (1707 images) and a test set (1000 images)

Task 1: Data dimensionality, distance-based classifiers

For the first part of this report the general purpose is to visualize the data using some dimensionality reduction techniques, develop a distance-based algorithm for classifying the hand-written digits of the previously mentioned data set and compare it to a different classifier which is again based on distance.

1.1 Procedure Description

For describing better the data set, it should be mentioned that each handwritten digit is represented by a vector of the 256-dimensional space. Classification involves finding the center of each digit's cloud of points in this space. A classifier implementing this method will be tested in 1.3. In this part, distances between cloud centers will be calculated to predict challenging digit pairs and classifier accuracy. To compute cloud centers, a 10-dimensional vector is initialized with zeros, and means of digit vectors are stored in it. Distances between centers are calculated, creating a 10x10 matrix indicating Euclidean distances between digit centers, helping identify similar digits.

A heat map of this matrix is the one displayed on the left of Figure 2. It shows how close the centers of each digit's cloud are, meaning that for colours closer to white, the specific digit is more probably to be wrongly identified as the digit that corresponds to this colour, whereas for colours closer to dark blue it is very unlikely for the digit to be wrongly identified as this. For example, 0 is most likely to be wrongly identified as 5 compared to the other numbers, as well as digit 4 is more likely to be wrongly identified as 9. Also, since the digit 1 seems to have the darkest colours, compared to the other digits, in its column, most probably will not have much wrong classifications.

Therefore, in general we expect the classifier of part 1.3 to have higher accuracy for digits that do not have centers near to other digits (like 1) and of course lower accuracy for those that have (like 2). Now, before checking whether indeed those expectations are valid by classifying the images of the test set, some dimensionality reduction algorithms will be inspected.

1.2 Dimensionality reduction algorithms

In this section three dimensionality reduction algorithms, PCA, U-MAP and T-SNE will be explored with some interesting 2D visualizations. Each one of them follow a different procedure in order to reduce the dimensions, but delving into a detailed analysis of their specific workings is beyond the scope of this report's objectives. In Figure 1 the visualizations for each of them are displayed.

PCA From the graph, is obvious that there is a lot of class overlap. The majority of the 10 classes are mixed, with only the class of the digit 1 to be distinguishable. This means that, PCA is not particularly effective for the specific high dimensional data set. However, from this plot, the expectation made in 1.1 that the digit 1 is going to be the most correctly classified digit, is confirmed.

U-MAP The U-MAP algorithm clearly classify the 10 different classes better than PCA. However, there are still some overlaps. For example, the classes of digits 4 and 7 overlap with that of 9. The classes of 8 and 3 also have some overlap as well as the ones of 4 and 1. The class of 1 again seems to be the most clearly classified class but also the class of 0 seems clearly classified.

T-SNE The graph shows that T-SNE is also better than PCA for the classification of digits. Compared to U-MAP, the same observations hold for T-SNE as well.

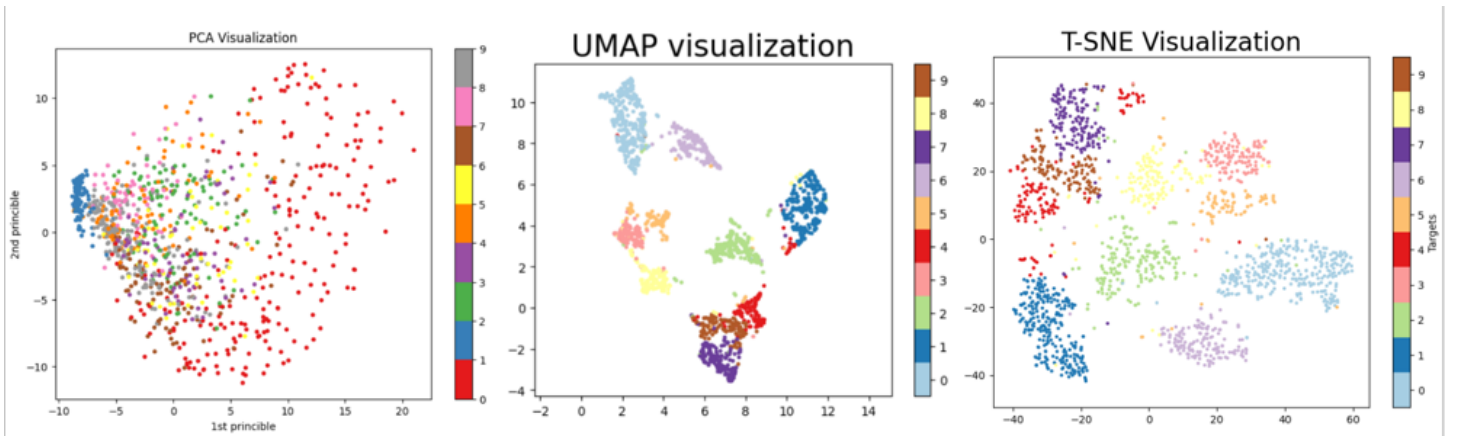


Figure 1: Dimensionality reduction algorithms visualizations

All in all, the visualisations of this section (except probably the PCA one) yield results which are mostly in line with the expectations made using the inter-class distances in 1.1.

1.3 Nearest Mean Classifier

In this part of the report the implementation of a Nearest mean classifier is performed in order to classify both train and test data and in addition to check whether our expectations hold. The classifier's class takes as input a data set containing the 256D vectors, a data set containing their respective digits and the calculated centers for the cloud of each of the 10 digits. After iterating through every vector of the first data set and compares it with all the 10 centers for each digit, it classifies the corresponding vector as the digit with the center of which it has the smallest distance compared to all the others.

Additionally, the classifier returns how many missclassifications were made for each digit as well as the percentages of them. Those are sorted starting with the digit that has the most missclassifications.

The Nearest Mean Classifier was run with both the training and the test data.

For the training data the accuracy obtained was approximately 86.4 %. The digit with the most missclassifications was 0 with approximately 20.6%. Then, the percentages for the other digits were(in descending order): 2: 15.0%, 4: 11.6%, 7: 11.2%, 8: 9.9%, 6: 9.4%, 5: 9.0%, 9: 8.6%, 3: 4.7%. The digit 1 had zero missclassifications.

For the test data the accuracy obtained was 80.4%. The digit with the most missclassifications was again 0 with approximately 23.5 %. Then the percentages for the other digits were(in descending order): 2:16.3%, 9:10.2%, 8:9.7%, 3:9.18%, 4:8.7%, 5:8.7%, 7:7.1%, 6:6.1%. Finally, the digit with the least missclassifications, as expected, is 1 with 0.51%.

1.4 K-NN classifier

K-NN (K-Nearest-Neighbor) distanced-based classifier was also tested with both the train and test set and compared to the nearest mean classifier of part 1.3. The chosen number for K, after the use of a validation set was 1 as it achieved the highest accuracy. So, the performance of the KNN algorithm using K=1 was evaluated on both the training and test data.

For both training and test data a classification report containing precision, recall, F1-score and support was created. For the training set, as expected all those percentages are very high between 98%-100%. The general classifier's accuracy is 99%. For the test set, the percentages are also relatively high which suggests that K-NN does a good job. The general classifier's accuracy is 91%. In more details, the highest f1-score is observed for the digit 1 and is 97%. The lowest is achieved by 5 and is 84%.

Finally, the performance of both classifiers on the test set was compared using the two confusion matrices on the right of Figure 2. For the Nearest Mean Classifier (NMC) it seems like the highest wrongly classified digit is 6 which is wrongly classified as 0 , 2.3% of times. Then, 8 is wrongly classifies as 2, 1.3% of times and 8 is wrongly classified as 0 with a percentage of 1%. The rest of the percentages on the matrix are below 1% and they are presented in Figure 2.

For the K-NN classifier the percentages of wrongly classified digits do not exceed the 0.7%. This was expected to be lower than for NMC because of the accuracy that was obtained which was higher for the K-NN classifier. The mostly wrongly classified digit is 3, which is wrongly classified as 8, 0.7% of times. Also zero is wrongly classified as 2, 0.5% of times. The rest of the percentages can also be found on Figure 2.

Overall, the results suggest that K-NN outperforms NMC, especially on the test dataset, as reflected in their respective accuracy rates and misclassification percentages.

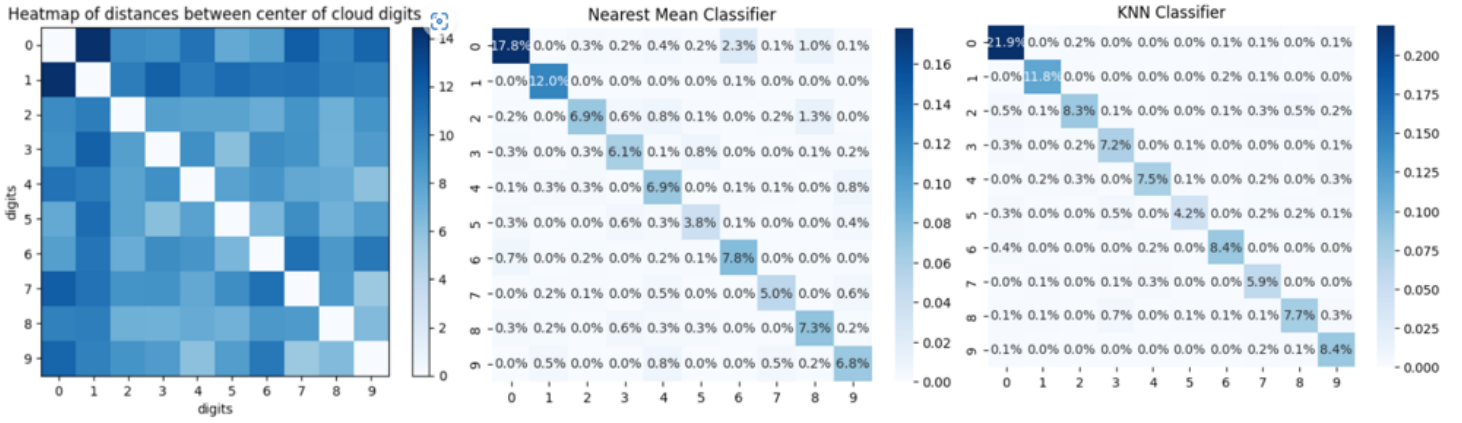


Figure 2: Heatmap and Confusion Matrices for the two classifiers

Task 2: Implement a multi-class perceptron algorithm

The aim of the second task is to implement from scratch a multi-class perceptron training algorithm and use it for training a single layer perceptron with 10 nodes (one per digit), each node having 256+1 inputs (inputs and bias) and 1 output.

Equation

Generalised Delta Rule:

$$\Delta W_{ji} = \eta \delta_j y_i \quad (1) \quad \delta_j = \begin{cases} \phi'(v_j)(d_j - y_j), & \text{if } j \text{ is an output node} \\ \phi'(v_j) \sum_{k \text{ of next layer}} \delta_k w_{kj}, & \text{if } j \text{ is a hidden node} \end{cases} \quad (2)$$

where:

w_{ji} : weight from node j to node i , η : learning rate, y_j : output of node j , v_j : activation of node j

Explanation

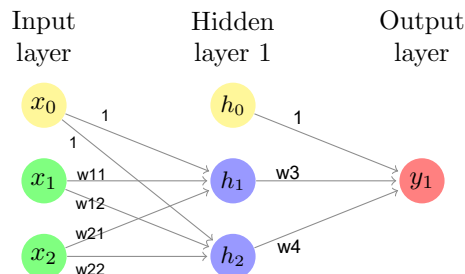
The code can be divided into sections. In the first section, the parameters were configured. More specifically, $num_classes = 10$, $learning_rate = 0.01$, $num_epochs = 250$. Then, the bias was integrated with the input by changing the shape of the input from (1707, 256) to (1707, 257) and the weights were initialized randomly. Following the training process, utilizing the delta rule (for output nodes) iteratively updates weights to minimize errors. The activation function used is sigmoid. Next, the performance of the perceptron is evaluated on both training and test set, with the scores 0.998 and 0.877 respectively.

According to the two algorithms examined in task 1 (NMC and K-NN), it can be concluded that the single-layer multi-class perceptron outperforms the NMC which has an accuracy of 86.4% and 80.4% on training and test datasets respectively, although it is slightly inferior to the K-NN algorithm in regards to the accuracy of the test dataset (91% K-NN vs 89% perceptron). As for the accuracy of the training dataset, they have the same accuracy which is almost 1 (99%).

Task 3: Implement the XOR network and the Gradient Descent Algorithm

For the last task, the XOR network was implemented and trained with the use of the gradient descent algorithm. More specifically, the generated XOR neural network was simulated with two inputs, two hidden nodes, and one output node, with biases nodes (equal to 1) for each non-input node.

3.1 XOR Neural Network Visualisation and Experiment Explanation



Initially, three functions were created: one for implementing the neural network, another for calculating the mean squared error in relation to the XOR problem, and the last one for updating the weights using the partial derivatives of the mean squared error. All three of these functions served as the foundation for the fourth function, which executes the XOR neural network from scratch, trains the network's weights through the gradient descent algorithm, and subsequently runs the neural network again with the updated weights.

During the training process, we adopted an exploratory approach by starting with randomly generated weights and experimenting with various activation functions and learning rates ('lazy approach'). Specifically, we tested not only the Sigmoid activation function but also explored the effectiveness of Tanh and ReLU. In terms of learning rates, we examined values such as 0.01, 0.1, and 0.2.

3.2 Findings

After experimenting with the various activation functions and learning rates, the results obtained after 10000 iterations (epochs) indicated the combination of the Sigmoid activation function with a learning rate of 0.2 as the most efficient. This combination leads to zero misclassified inputs and a mean squared error (MSE) of 0.0029. Irrespective of the learning rate, the Sigmoid activation function demonstrated the best performance. As for the two other activation functions, the Tanh and ReLU, they exhibited similar and relatively poorer performance, with a minor improvement when using a smaller learning rate of 0.01.

The Sigmoid activation function's superior performance can be attributed to its ability to simulate non-linear transformations, the preservation of a smooth and differentiable gradient, and its suitability for the relatively simple binary classification task of XOR (as it limits output values to the range of $[0, 1]$). The Tanh activation function, on the contrary, although sharing similarities with Sigmoid, may perform poorly because of its sensitivity to vanishing gradients. Finally, ReLU is more suitable for complex problems, which might explain its suboptimal performance in that case. For the ReLU activation case, it might be the specific random seed for generating numbers used in the program. Maybe the majority of the nine generated numbers for weights would be negative numbers and thus zero for the ReLU activation function.

Contributions

Myriana: Task 1 code and report

Elena: Task 2 code and report

Pelagia: Task 3 code and report

References

1. Principal Component Analysis (PCA) in Python Tutorial
<https://www.datacamp.com/tutorial/principal-component-analysis-in-python>
2. How to Use UMAP
https://umap-learn.readthedocs.io/en/latest/basic_usage.html
3. t-SNE Decomposition on MNIST Dataset
<https://www.kaggle.com/code/venkatkrishnan/t-sne-decomposition-on-mnist-dataset>
4. k-Nearest Neighbor classification
<https://customers.pyimagesearch.com/lesson-sample-k-nearest-neighbor-classification/>
5. Neural Network xor example from scratch
<https://www.codingame.com/playgrounds/59631/neural-network-xor-example-from-scratch-no-libs>
6. Draw a Graph Using LaTeX
<https://www.baeldung.com/cs/latex-drawing-graphs>
7. Activation Functions In Python
<https://www.nbshare.io/notebook/751082217/Activation-Functions-In-Python/>